

2.8 Transforming API response in Spring Web MVC

Why do we need a structured ApiResponse?

When building *REST APIs*, *consistency* in *responses* is critical for clients (frontend, mobile apps, integrations). If every *endpoint* and *exception* returns data in a *different format*, clients need to handle *multiple parsing logics* → leading to *confusion* and *bugs*.

Problems with unstructured responses:

- Some endpoints return *plain objects*, others *wrap them*.
- *Error responses* differ (sometimes *plain text*, sometimes *Spring's default error JSON*).
- Clients need "*if-else*" checks for *each* type of *response*.

Benefits of a structured format:

- *Consistent schema* for both *success* and *error* responses.
- *Easier* to *parse* and *display* in client applications.
- *Centralized error handling* → no leaking *stack traces* to clients.
- Better *developer experience* (DX) → reduces guesswork for consumers.

Designing the ApiResponse (Generic wrapper for every API response):

ApiResponse: Fixed structure for every response (error | data).

```
@Data
public class ApiResponse<T> {
    @JsonFormat(pattern = "hh:mm:ss dd-MM-yy")
    private LocalDateTime timestamp;
    private String message;
    private T data;
    private ApiError error;
    ApiResponse(String message) {
        this.timestamp = LocalDateTime.now();
        this.message = message;
    }
    ApiResponse(T data, String message) {
        this(message);
        this.data = data;
    }
    ApiResponse(ApiError error, String message) {
        this(message);
        this.error = error;
    }
}
```

If there will any *exception occurs* then that will be stored in *error* and *data* field will be *null*. Similarly, if *data fetched successfully* then *error* field will be *null* and fetched *data* will be stored in *data*.

Different *controllers* return *different types* of data. So, I create *data field* as *generic type T* to store *any type* of controller response in that field.

ApiError: Encapsulates *error details*.

```
@Builder
@Data
public class ApiError {
    private HttpStatus status;
    private String message;
    private List<String> errors;
}
```

I am storing *errors* in a *list* because *MethodArgumentNotValidException* exception throws *multiple exceptions* at *once* (id not valid, age not valid, email not valid, ...).

GlobalExceptionHandler: Handling Exceptions *Globally*. Instead of writing *try-catch* in every controller, use `@RestControllerAdvice` + `@ExceptionHandler`.

`@RestControllerAdvice`

```
public class GlobalExceptionHandler {

    // Handles No-Such-Element-Found exception
    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<ApiResponse<?>> HandelNoSuchElementException(NoSuchElementException exception) {
        ApiError error = ApiError.builder()
            .status(HttpStatus.NOT_FOUND)
            .message(exception.getClass().getName())
            .errors(List.of(exception.getMessage()))
            .build();
        ApiResponse<?> response = new ApiResponse<>(error, "Not Found !!");
        return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
    }

    // Handles every generic exceptions ( triggers only if a handler not present )
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<?>> HandelInternalServerError(Exception e) {
        ApiError error = ApiError.builder()
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .message(e.getClass().getName())
            .errors(List.of(e.getMessage()))
            .build();
        ApiResponse<?> response = new ApiResponse<>(error, "Something went wrong !!");
        return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR);
    }

    // handles validation exceptions
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<?>> HandleInputValidationError(MethodArgumentNotValidException e) {
        List<String> errors = new LinkedList<>();
        e.getBindingResult()
            .getAllErrors()
            .forEach(err -> errors.add(err.getDefaultMessage()));
        ApiError error = ApiError.builder()
            .status(HttpStatus.BAD_REQUEST)
            .message(e.getClass().getName())
            .errors(errors)
            .build();
        ApiResponse<?> response = new ApiResponse<>(error, "Input Validation Exception !!");
        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }

    // Handles exceptions user input type mismatch in path-variable or request-params
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiResponse<?>> HandelMethodArgumentTypeMismatchError(MethodArgumentTypeMismatchException e) {
        ApiError error = ApiError.builder()
            .status(HttpStatus.BAD_REQUEST)
            .message(e.getClass().getName())
            .errors(List.of(e.getMessage()))
            .build();
        ApiResponse<?> response = new ApiResponse<>(error, "MethodArgumentTypeMismatchException");
        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }
}
```

`@ExceptionHandler` → It collects *exceptions* of a controller and *give control* to the *annotated method*.

`@RestControllerAdvice` → It makes *ExceptionHandler* annotation so powerful that can *collect exceptions* from every controller not only for one.

GlobalResponseHandler: Wrapping All Successful Responses Globally.

- Sometimes you return `ResponseEntity<EmployeeDTO>` from a controller. Without intervention, the client receives just the raw `EmployeeDTO`.
- We want **all responses** to be **wrapped** in `ApiResponse`.
- Spring Boot provides `ResponseBodyAdvice<T>` for this.

@RestControllerAdvice

```
public class GlobalResponseHandler implements ResponseBodyAdvice<Object> {  
    @Override  
    public boolean supports(MethodParameter returnType, Class<? extends HttpMessageConverter<?>> converterType) {  
        return true;  
    }  
    @Override  
    public Object beforeBodyWrite(Object body,  
        MethodParameter returnType,  
        MediaType selectedContentType,  
        Class<? extends HttpMessageConverter<?>> selectedConverterType,  
        ServerHttpRequest request,  
        ServerHttpResponse response) {  
  
        if (body instanceof ApiResponse<?>) return body; // Avoiding double wrap.  
        return new ApiResponse<>(body, "Successful");  
    }  
}
```

Final Flow:

- **Success case:** `EmployeeDTO` → `ApiResponse<EmployeeDTO>` (with message: **"Successful"**).
- **Validation error:** `MethodArgumentNotValidException` → `ApiResponse<ApiError>` (with detailed validation messages).
- **Unhandled exception:** `Exception` → `ApiResponse<ApiError>` (with **500** status).

Success case	Exception Case
<pre>// http://localhost:8080/employee/1 { "timestamp": "10:27:34 28-08-2025", "message": "Successful", "data": { "id": 1, "name": "Sibasundar Jena", "luckyNumber": 2, "email": "sibasundarj8@gmail.com", "salary": 35000.54, "age": 22, "active": true, "role": "USER", "dateOfJoining": "2001-04-24" }, "error": null }</pre>	<pre>{ "timestamp": "07:47:36 28-08-2025", "message": "Input Validation Exception !!", "data": null, "error": { "status": "BAD_REQUEST", "message": "MethodArgumentNotValidException", "errors": ["must be greater than or equal to 9999.99", "active must be true", "luckyNumber must be a Prime", "Employee role must be either USER or ADMIN", "Employee must be an adult(at-least 18)", "must be a date in the past or in the present", "Enter valid email"] } }</pre>

Takeaway:

- Use `ApiResponse` for *consistent schema*.
- Use `GlobalExceptionHandler` for **errors**.
- Use `ResponseBodyAdvice` for **success responses**.
- This ensures that all **endpoints**, regardless of **success** or **failure**, return **responses** in the **same consistent format**, maintaining a **reliable API contract**.